EcoJudge - System Architecture

System Architecture: Eco-Loop Building Agents — Adversarial Setpoint Court

Tool-Calling Architecture

Two paths expose tools, for different reasons:

- Direct Python function calls (src/agents.py, src/judge.py, src/ep_bridge.py, src/orchestrator.py) drive the actual per-block debate loop. Chosen over routing every call through the MCP transport because the loop is latency-sensitive (2 LLM calls x 4 blocks x N iterations) and needs deterministic, in-process error handling for the self-repair retry.

- src/mcp_server.py wraps the read-side (metrics, errors, carbon signal) as real MCP tools via FastMCP (stdio transport), satisfying the "MCP Server or custom agentic tools" requirement literally and demonstrating the tool-calling contract independent of the orchestrator's internals.

Prompt Engineering Strategy

Each persona (Energy Advocate, Comfort Advocate) gets a fixed system prompt defining its sole incentive and a strict "JSON only" output contract. The user message carries only the current block's setpoints and last-measured metrics — no simulation logs, no history — keeping prompts small and the JSON-parsing surface narrow. A regex extracts the first {...} block from the response before json.loads, tolerating models that wrap JSON in markdown fences.

Prompt Latency Management

The precedent cache (src/precedent_cache.py) is the primary latency lever: once a (block, temp_bucket, pmv_bucket, carbon_level) state has been debated once, later iterations reuse that verdict with zero LLM calls. This also reduces exposure to API flakiness over a long run — fewer calls means fewer chances for a timeout to interrupt the loop. The cache is independent of the self-repair path below: a simulation error is a property of the whole patched schedule (all 4 blocks applied together), not of one cached block-verdict, so an error doesn't invalidate any specific cache entry — it triggers the separate repair flow instead.

Handling Simulation Output/Logs

EnergyPlus's .err file is scanned line-by-line for Severe / Fatal markers only — the full file (which can run to hundreds of lines even on clean runs, full of Warning and informational lines) is never fed to the LLM. The .csv output (via -r/--readvars) is read with csv.DictReader and reduced to four numbers per block (avg temp, avg RH, avg PMV, kWh) before anything touches the LLM context window. One real gotcha found during integration testing: EnergyPlus's CSV meter columns can carry a trailing space in their header ("Electricity:Facility J (Hourly) ") when the Output:Meter object is added programmatically via eppy rather than hand-edited — the CSV reader strips header whitespace defensively rather than assuming an exact column-name match.

Self-Correction Loop

On a severe/fatal EnergyPlus error, the failing setpoints and the exact error text are handed to a dedicated Repair Agent persona (src/agents.py::propose_repair), one call per block, which proposes corrected setpoints. The repaired schedule is patched in and the simulation retried once. If the retry succeeds, its metrics feed the normal debate for that iteration as if nothing had gone wrong. If it still fails, the loop falls back to the last known-good setpoints and skips the debate for that round rather than debating from data that doesn't exist. Both branches are covered by tests/test_orchestrator.py against a mocked EnergyPlus bridge, since reliably forcing a genuine EnergyPlus severe error on demand isn't practical — our real runs against the bounded 14-32C setpoint range haven't triggered one.

Honest Scope Note

Control granularity here is 4 repeating daily time blocks (night/morning/afternoon/evening), refined iteratively across full re-simulations of a 3-day period — not live per-hour intra-simulation callbacks. EnergyPlus subprocess invocations don't carry thermal state between runs; true real-time coupling would need EnergyPlus's embedded Python Plugin/EMS system (a separate Python 3.8 runtime bundled with the engine), which was assessed as too high-risk to get working reliably in the time available. This is documented here rather than glossed over in the demo video.

Comfort is measured via ISO 7730 PMV (pythermalcomfort, pinned to 2.10.0 — later versions crash on import in this environment due to a numba disk-cache bug), computed with fixed assumptions (mean radiant temp = air temp, air velocity 0.1 m/s, metabolic rate 1.2 met, clothing 0.5 clo) since the simulation has no separate radiant-temperature or occupant-activity sensors.

Novelty Positioning

No individual mechanism in this system is unpublished: multi-agent debate + judge arbitration is a common pattern (including as a tutorial toy example), LLM-writes-and-repairs-control-code is Agentic Policy Search's territory (arXiv 2501.19340), and precedent/case-based caching for LLM agents is its own published subfield. What's being submitted is a specific combination applied end-to-end against a live EnergyPlus instance, not a claim of an unpublished technique.